

Programming with Python – CHEATSHEET

Sean McGowan



Getting started

PYTHON-ENVIRONMENT

Interface

- REPL: execute code
- Editor: editing scripts/notebooks

Useful commands

- Check version: `python --version`
- List objects: `dir()`

FILES

- Run script: in VSCode press Run
- Run selected lines: `<shift> + <enter>`
- Comment code: `# a comment`
- Code cell: `#%% a code cell`

Comments explain *why* not *what*

Objects

TYPES

- Numeric, logical, character
- Check type: `type(x)`

Assign variables: `x = 1, y = True, z = "a"`

COMMON OBJECTS

- Strings: `"Hello world!"`
- Lists: `[1,2,3]`
- Tuples: `(1, "abc", [1,2,3])`
- Dictionaries: `{'x': [1,2,3], 'y': "Hello"}`

Modify and copy lists:

`append()`, `extend()`, `copy()`

OPERATORS

- Arithmetic: `+`, `-`, `*`, `/`, `**`
`.*`, `.-`, `.*`, `./`, `.**` for element-wise
- Updating: `+=`, `-=`, `*=`, `/=`
- Logical: `<`, `>`, `<=`, `>=`, `and`, `or`,
`==` (equals), `!=` (not equals)

INDEXING

- Starts at 0
- Element: `x[i]`
- Range: `x[start:stop:step]`, starting at start, ending before stop, incrementing by step
- Tuple unpacking: `a,b,c = T`
- Dictionaries: `D['key']`

Control flow

STRUCTURES

If / else: branched decision making

```
if x < 0:
    ...
elif x == 0:
    ...
else:
    ...
```

For: iterate over sequences

```
for i in 1:5:
    ...
```

While: loop until condition is false

```
while x < 1:
    ...
```

Always *preallocate* objects before loops

Nested loops: loops within loops

```
for i in 1:4:
    for j in 1:4:
        if i*j == 4:
            ...
```

Loop control

- `break`: stop loop
- `continue`: skip iteration

List comprehension: list from existing list
`[expression for item in sequence if condition == True]`

Functions

DEFINING FUNCTIONS

Function block:

```
def f(x,y):
    return x+2*y
```

Anonymous functions: no-name functions within functions

```
lambda x: x**2
```

Multiple outputs: return a tuple

USING FUNCTIONS

Calling

- Positional: `f(1,2)`
- Named: `f(x=1, y=2)`

Defaults: arguments with default values

```
def g(a,b,n=2):
    return a**n-b**n
```

Variable scope: local, global, nonlocal

Files: read ('r'), write ('w'), append ('a')
with `open('file', 'r')` as `f`:

USEFUL FUNCTIONS

- `round()`, `math.floor()`, `math.ceil()`
- `abs()`, `math.copysign(1,())`
- `math.sqrt()`, `math.exp()`, `math.log()`, `math.log10()`
- `math.sin()`, `math.cos()`, `math.tan()`, `math.sinh()`, `math.cosh()`, `math.tanh()`, `math.asin()`, `math.acos()`, `math.atan()`

Packages

USING PACKAGES

- Install: `pip install scipy`
- Import full package: `import scipy`
- Import specific functions: `from math import sin, cos, pi`
- Alias: `import numpy as np`

POPULAR PACKAGES

- Plotting: `matplotlib`, `seaborn`, `plotly`
- Math: `numpy`, `scipy`, `sympy`
- Stats: `statsmodels`, `pymc`

Debugging and help

Common errors

- `IndexError`: index out of range
- `TypeError`: wrong type
- `ZeroDivisionError`: divide by zero
- `NameError`: variable/function not defined
- `ValueError`: inappropriate argument

Debugging tools

- Error message and traceback
- Use `print()` to follow intermediate values
- `pdb` module: step through code

Documentation and other resources

- `help()`: view documentation for functions, modules, packages
- `dir()`: list attributes of object
- Online resources: Python documentation, Stack Overflow, Python 101 by Driscoll